TEMA 03

Android Studio y Jetpack Compose

DAM · Programación Multimedia y Dispositivos Móviles

Objetivos y resultados de aprendizaje

- Crear un proyecto Android en Kotlin utilizando la plantilla Empty Activity (Jetpack Compose).
- Reconocer los paneles y archivos esenciales de Android Studio.
- Configurar y utilizar un dispositivo virtual para ejecutar y probar aplicaciones.
- Comprender el ciclo de arranque de una Activity y la función setContent.
- Crear interfaces con funciones @Composable y gestionar su estado.
- Separar la validación de datos de la presentación visual.
- Realizar pruebas sistemáticas y depurar errores de lógica.

1. Crear el proyecto Tema03compose

Abre Android Studio. Desde la pantalla de bienvenida pulsa New Project. Si hay un proyecto abierto, selecciona File → New → New Project. En Phone and Tablet elige Empty Activity, la plantilla que usa Kotlin y Jetpack Compose. No elijas Empty Views Activity: esa plantilla emplea el sistema clásico de vistas XML.

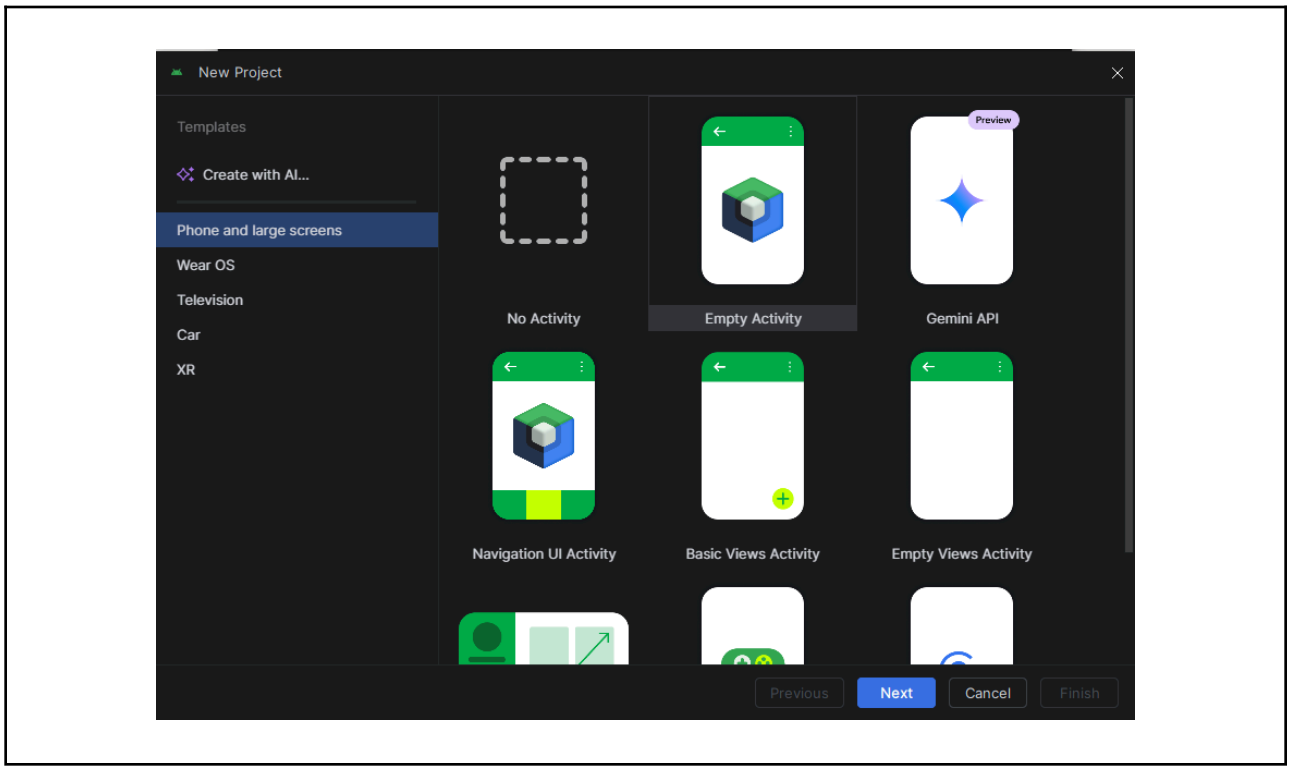


Figura 1. Selección de la plantilla Empty Activity.

1. En Name escribe Tema03compose.
2. Comprueba que el Package name sea com.example.tema03compose.

3. Selecciona Kotlin si el asistente permite escoger lenguaje.
4. Elige una ubicación de proyecto sencilla. Evita rutas con caracteres especiales en carpetas de usuario si tu equipo ha tenido problemas con Gradle.
5. Pulsa Finish y espera a que finalice la sincronización Gradle. No cierres Android Studio mientras se descargan dependencias.

La versión de Android Studio puede cambiar la disposición de estos campos. Lo importante es conservar el nombre del proyecto y usar Compose.

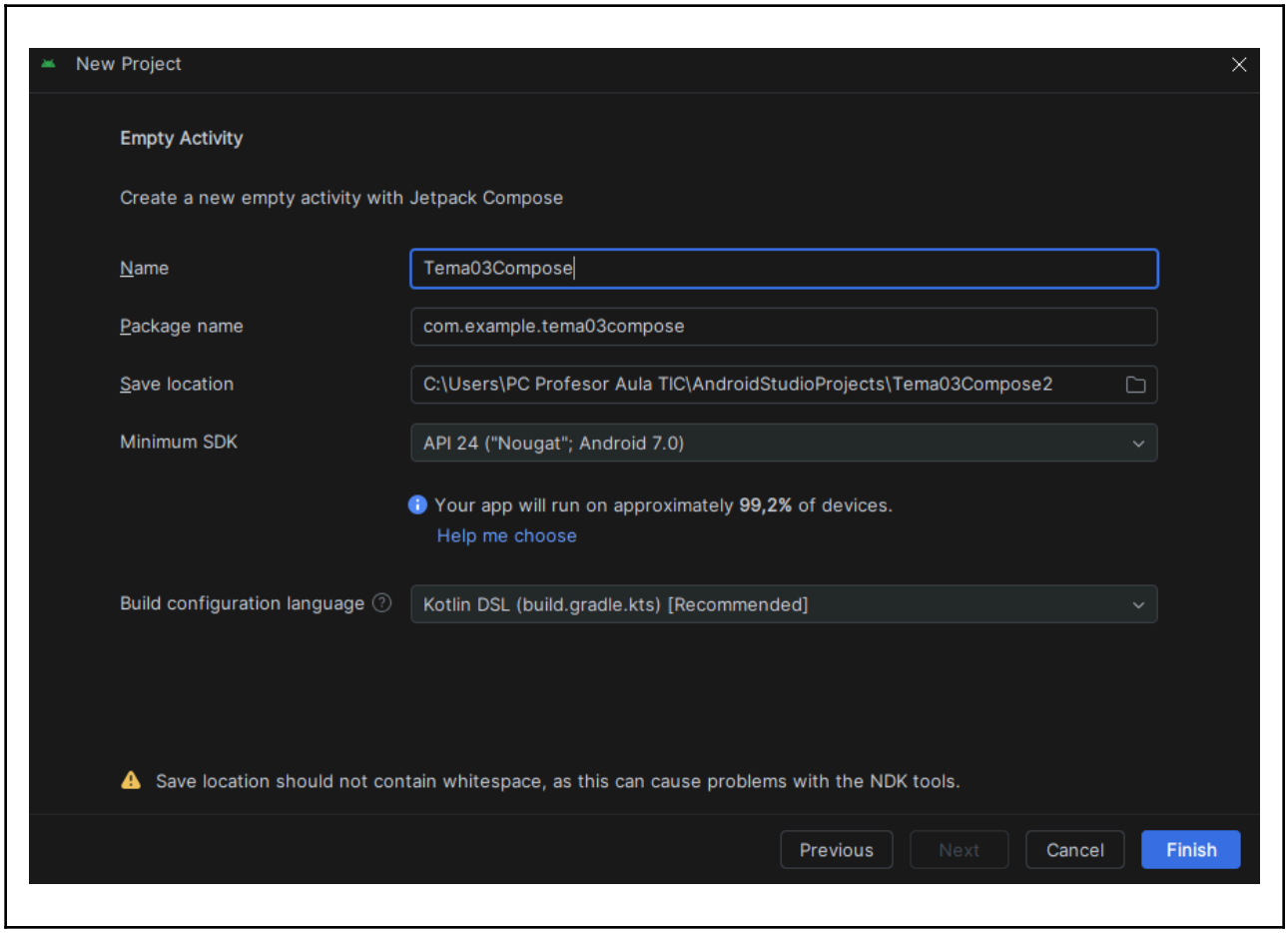


Figura 2. Configuración del proyecto Tema03compose.

2. Reconocer la interfaz de Android Studio

El panel Project muestra los archivos. El editor central permite modificar código Kotlin. En la barra superior aparecen el dispositivo seleccionado, la configuración app y el botón Run (triángulo verde). En la zona inferior pueden aparecer Build, Logcat y otras herramientas.

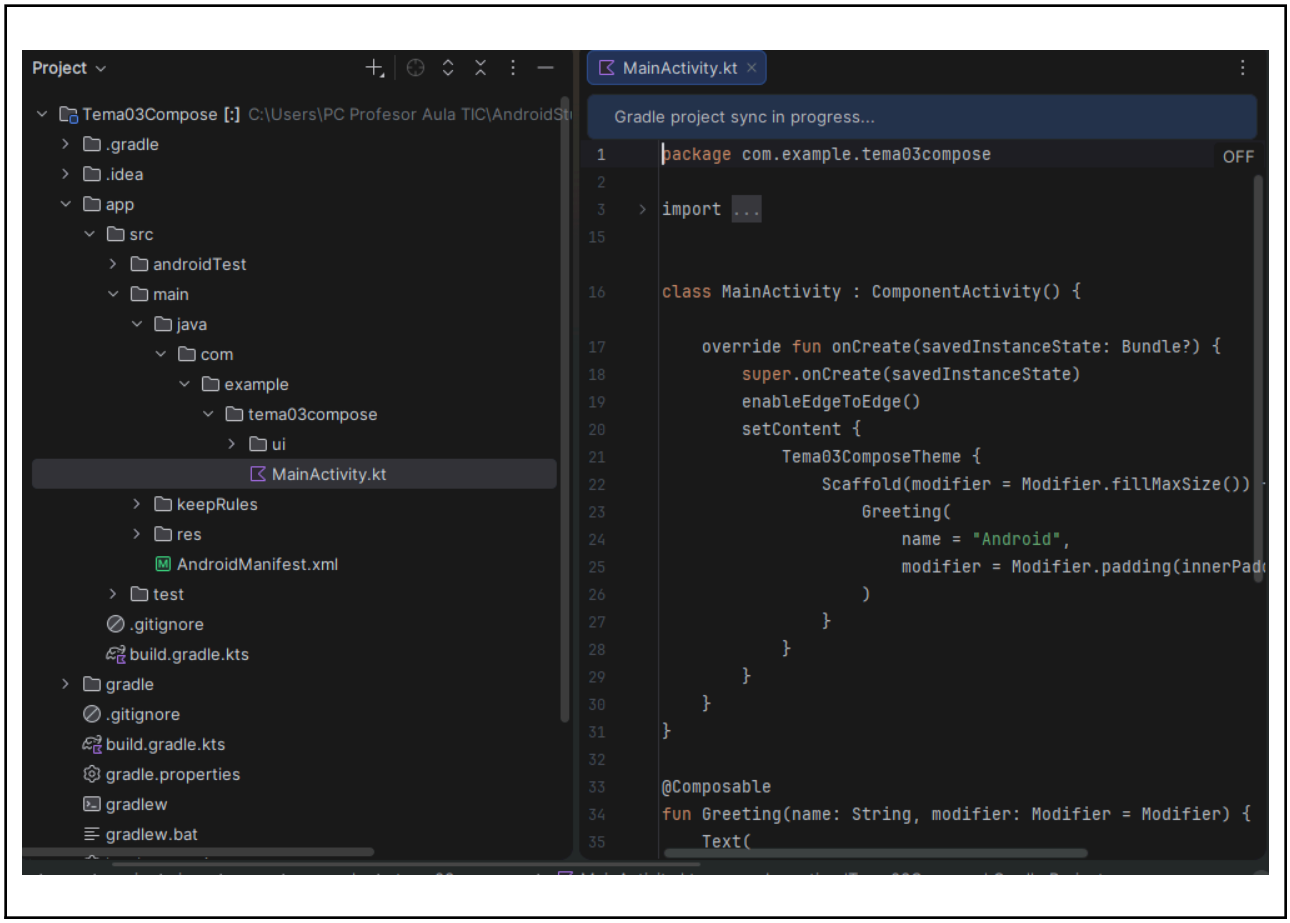


Figura 3. Pantalla principal de Android Studio.

2.1. Archivos fundamentales

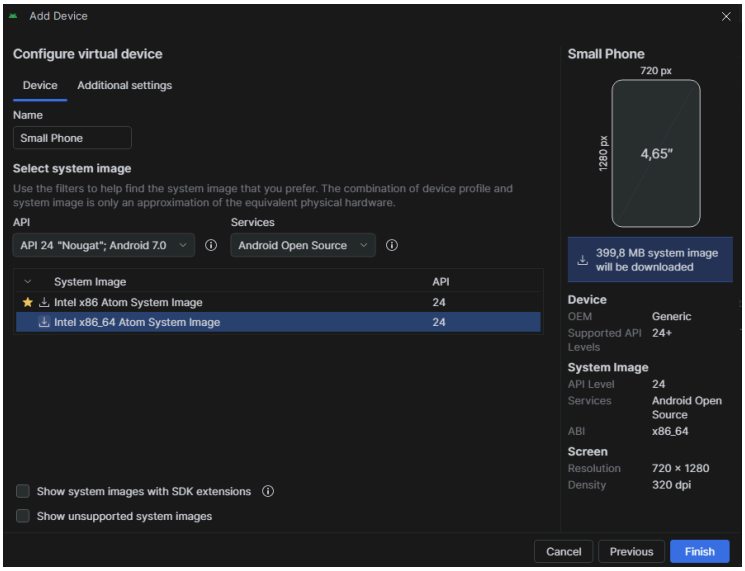
- app: módulo principal de la aplicación.
- manifests → AndroidManifest.xml: declara componentes y configuración esencial.
- kotlin+java → com.example.tema03compose: paquete donde escribiremos MainActivity.kt y ValidarAcceso.kt.
- res: recursos como iconos y cadenas, según la plantilla.
- Gradle Scripts: configuración de compilación y dependencias.

MainActivity.kt contiene el punto de entrada y la interfaz. ValidarAcceso.kt contendrá una función Kotlin independiente que decide si se permite el acceso.

3. Preparar y ejecutar el emulador

En Tools → Device Manager se gestionan los dispositivos virtuales (AVD). Si ya existe uno funcional, reutilízalo. En la práctica real se utilizó un dispositivo Small Phone con Android 7.0, API 24, en un equipo de recursos limitados.

6. Abre Device Manager y comprueba si tienes un dispositivo creado.
7. Si no existe, selecciona Create Device, elige un teléfono y una imagen de sistema compatible con tu ordenador.
8. Arranca el emulador y espera a que aparezca la pantalla de Android.
9. En la barra superior selecciona ese dispositivo, asegúrate de que la configuración es app y pulsa Run.
10. Espera a que Gradle compile e instale la aplicación.



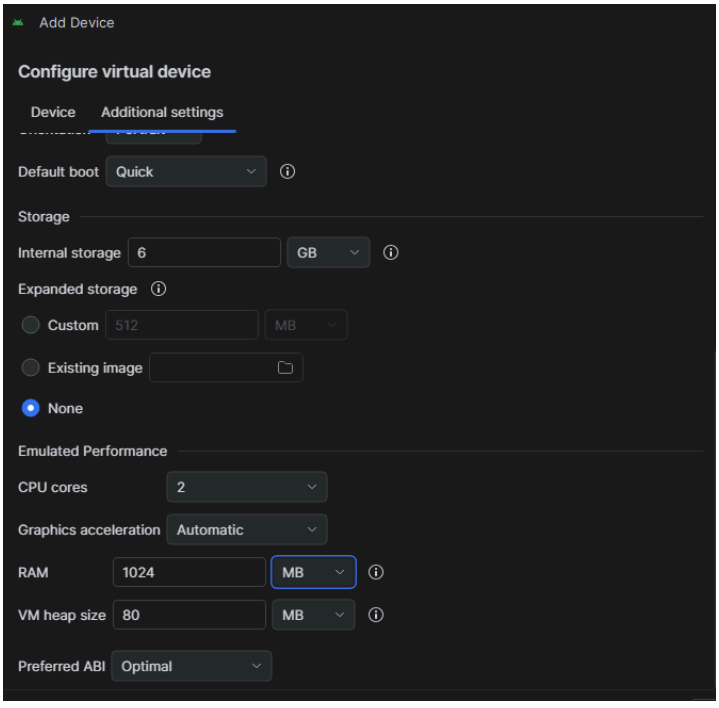


Figura 4. Device Manager con el emulador.

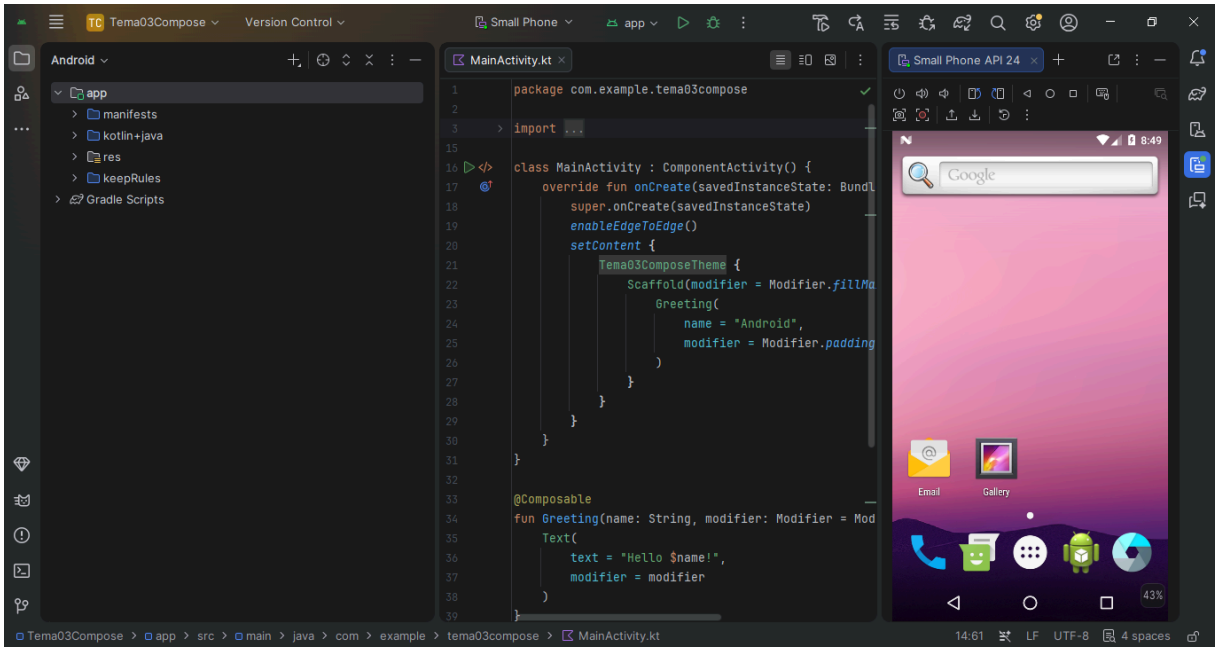


Figura 5. Aplicación ejecutándose en el emulador.

Si aparece un error, revisa la pestaña Build. Si la aplicación se cierra, utiliza Logcat. No intentes ejecutar MainActivity.kt como una función main() de consola.

4. El punto de entrada: MainActivity.kt

En una aplicación Compose, MainActivity es una clase que hereda de ComponentActivity. Android invoca onCreate cuando crea la Activity. setContent define la interfaz visible mediante funciones composables.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Tema03composeTheme {
                PantallaAcceso()
            }
        }
    }
}
```

La llamada a PantallaAcceso() indica qué pantalla queremos mostrar. Tema03composeTheme aplica el tema visual del proyecto. Los nombres deben coincidir exactamente con los definidos en el código.

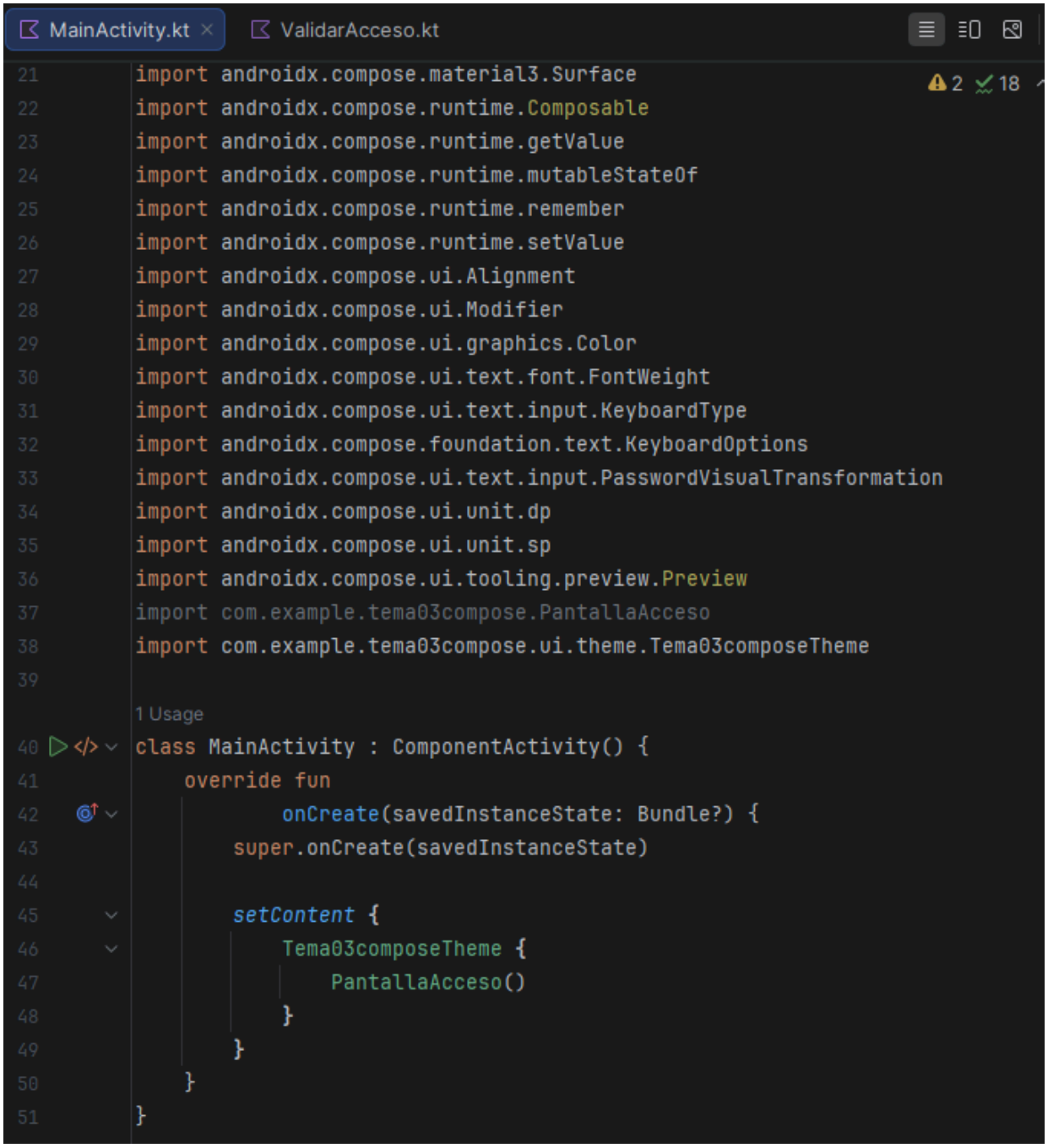


Figura 6. MainActivity.kt: onCreate y setContent.

5. Introducción a Jetpack Compose

Jetpack Compose permite crear interfaces declarativas con Kotlin. En vez de indicar paso a paso cómo cambiar cada control, describimos cómo debe verse la pantalla para el estado actual.

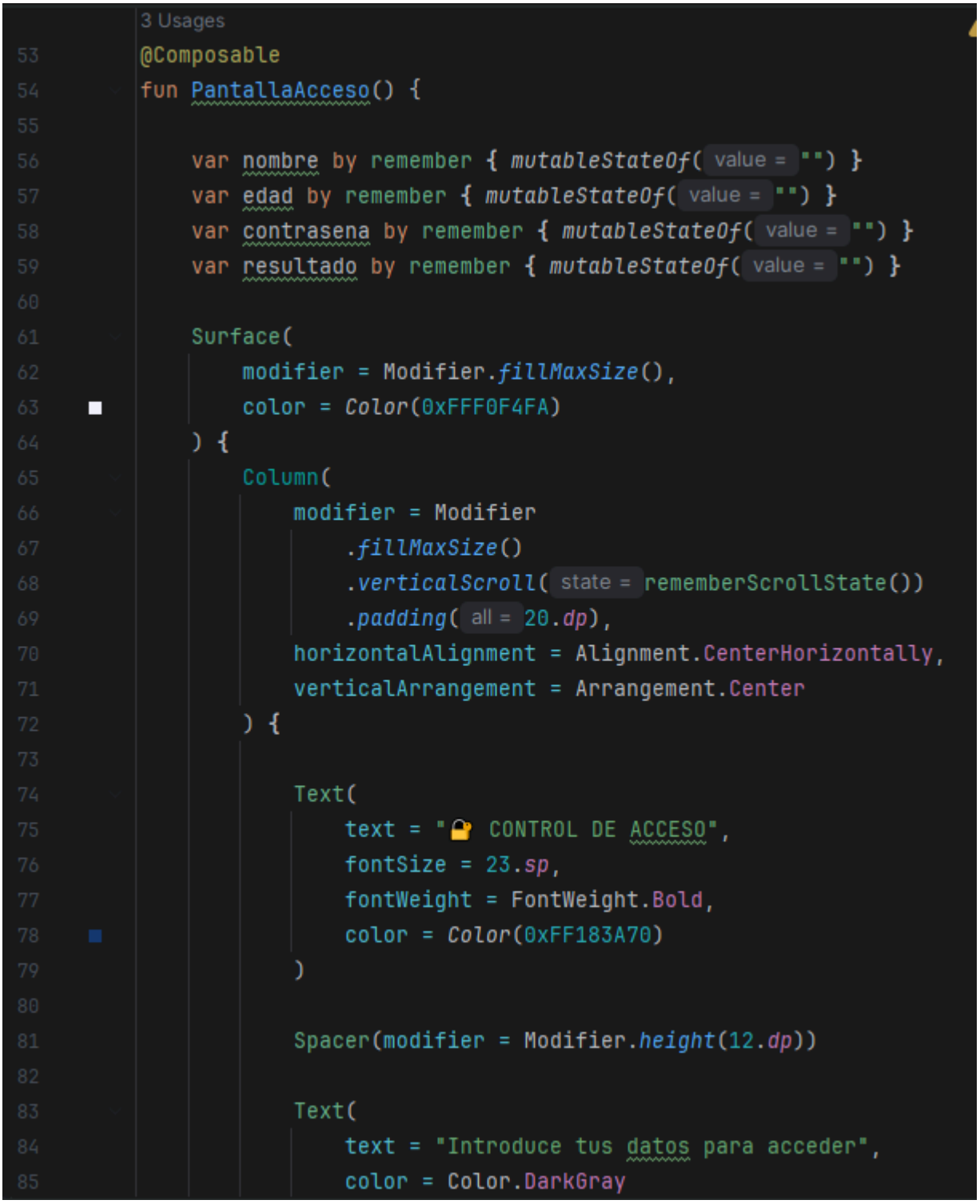
5.1. Componentes esenciales

- @Composable: identifica una función que dibuja parte de la interfaz.
- Text: muestra un texto.
- Column: coloca componentes uno debajo de otro.
- Surface: aporta una superficie y su color de fondo.
- Card: agrupa elementos dentro de una tarjeta.
- Spacer: añade separación entre componentes.
- OutlinedTextField: campo de entrada de texto.
- Button: botón que responde a pulsaciones.
- Modifier: define tamaño, márgenes, desplazamiento y otros aspectos.

5.2. Estado con remember y mutableStateOf

```
@Composable
fun PantallaAcceso() {
    var nombre by remember { mutableStateOf("") }
    var edad by remember { mutableStateOf("") }
    var contrasena by remember { mutableStateOf("") }
    var resultado by remember { mutableStateOf("") }
    // Aquí se añaden los componentes de la interfaz.
}
```

remember conserva el valor durante las recomposiciones. mutableStateOf permite que Compose observe los cambios. Cuando el usuario escribe, la variable cambia y la interfaz puede actualizarse. El código usa la delegación by, que requiere sus importaciones Kotlin correspondientes.



```
3 Usages
@Composable
fun PantallaAcceso() {

    var nombre by remember { mutableStateOf( value = "") }
    var edad by remember { mutableStateOf( value = "") }
    var contrasena by remember { mutableStateOf( value = "") }
    var resultado by remember { mutableStateOf( value = "") }

    Surface(
        modifier = Modifier.fillMaxSize(),
        color = Color(0xFFF0F4FA)
    ) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .verticalScroll( state = rememberScrollState())
                .padding( all = 20.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {

            Text(
                text = "🔐 CONTROL DE ACCESO",
                fontSize = 23.sp,
                fontWeight = FontWeight.Bold,
                color = Color(0xFF183A70)
            )

            Spacer(modifier = Modifier.height(12.dp))

            Text(
                text = "Introduce tus datos para acceder",
                color = Color.DarkGray
```

Figura 7. Estados de PantallaAcceso.

6. Construir el formulario

Organizaremos el contenido dentro de una Surface y una Column. Con fillMaxSize ocupamos el espacio disponible; con padding añadimos márgenes. verticalScroll permite desplazarse cuando el teclado reduce el espacio visible.

```
Surface(
    modifier = Modifier.fillMaxSize(),
    color = Color(0xFFF0F4FA)
) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .verticalScroll(rememberScrollState())
            .padding(20.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text("🔐 CONTROL DE ACCESO", fontSize = 23.sp)
        Text("Introduce tus datos para acceder")
        // Tarjeta y campos del formulario
    }
}
```

En Kotlin, las unidades dp sirven para dimensiones y sp para tamaños de texto. La clase Color recibe aquí un valor hexadecimal.

6.1. Campo Nombre

```
OutlinedTextField(
    value = nombre,
    onValueChange = { nombre = it },
    label = { Text("Nombre") },
    modifier = Modifier.fillMaxWidth(),
    singleLine = true
)
```

value enlaza el campo con la variable nombre. onValueChange recibe el texto nuevo y lo guarda. label muestra la etiqueta; singleLine evita saltos de línea.

6.2. Campo Edad

```
OutlinedTextField(
    value = edad,
    onValueChange = { edad = it },
    label = { Text("Edad") },
    keyboardOptions = KeyboardOptions(
        keyboardType = KeyboardType.Number
    ),
    modifier = Modifier.fillMaxWidth(),
    singleLine = true
)
```

KeyboardType.Number solicita un teclado numérico, pero no garantiza que el contenido sea una edad válida. La validación debe realizarse igualmente en Kotlin.

6.3. Campo Contraseña

```
OutlinedTextField(
    value = contrasena,
    onValueChange = { contrasena = it },
    label = { Text("Contraseña") },
    visualTransformation = PasswordVisualTransformation(),
```

```
    modifier = Modifier.fillMaxWidth(),
    singleLine = true
)
```

PasswordVisualTransformation oculta visualmente los caracteres. No es un sistema de cifrado ni de autenticación real: esta aplicación es un ejercicio didáctico.

7. Separar la lógica: ValidarAcceso.kt

En el panel Project haz clic derecho sobre com.example.tema03compose → New → Kotlin Class/File → File. Escribe ValidarAcceso. Comprueba que el archivo está en el mismo paquete que MainActivity.kt.

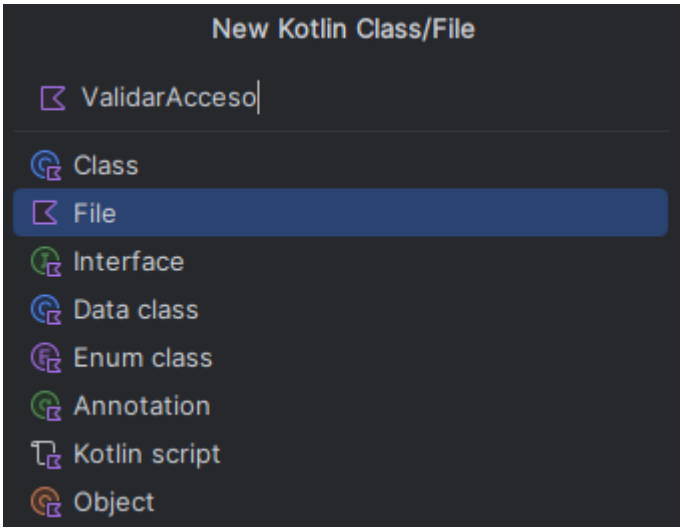


Figura 10. Archivo ValidarAcceso.kt en el proyecto.

La función recibe tres cadenas y devuelve un mensaje. No contiene componentes visuales, por lo que puede comprobarse independientemente de la interfaz.

```
package com.example.tema03compose

fun validarAcceso(
    nombre: String,
    edadTexto: String,
    contrasena: String
): String {
    val nombreLimpio = nombre.trim()

    if (nombreLimpio.isBlank()) {
        return "Acceso denegado: introduce tu nombre"
    }

    val edad = edadTexto.toIntOrNull()
    if (edad == null || edad < 0) {
        return "Acceso denegado: edad no válida"
    }

    if (edad < 18) {
        return "Acceso denegado: eres menor de edad"
    }

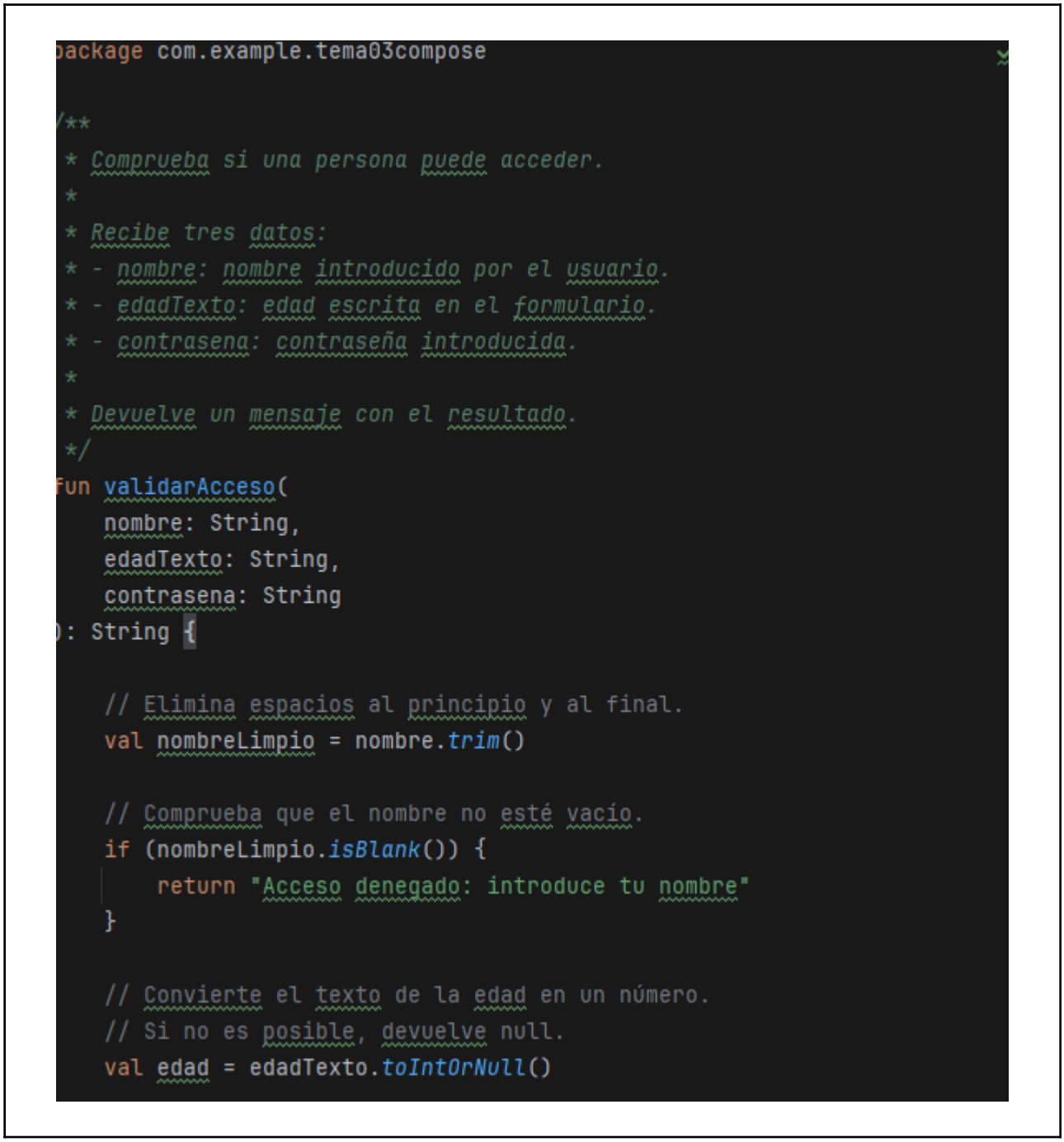
    if (contrasena.length < 8) {
        return "Acceso denegado: contraseña demasiado corta"
```

```
    }

    return "Acceso permitido. Bienvenido/a, $nombreLimpio"
}
```

7.1. Explicación de las instrucciones

- trim(): elimina espacios al principio y al final del nombre.
- isBlank(): detecta cadenas vacías o compuestas solo por espacios.
- toIntOrNull(): convierte un texto en número entero o devuelve null si no puede.
- edad == null || edad < 0: rechaza datos que no representan una edad válida.
- edad < 18: impide el acceso a menores de edad.
- contrasena.length < 8: exige una longitud mínima de ocho caracteres.
- return: termina la función y entrega el mensaje al código que la llamó.
- $nombreLimpio: inserta el nombre en el texto mediante interpolación.



```
package com.example.tema03compose

/**
 * Comprueba si una persona puede acceder.
 *
 * Recibe tres datos:
 * - nombre: nombre introducido por el usuario.
 * - edadTexto: edad escrita en el formulario.
 * - contrasena: contraseña introducida.
 *
 * Devuelve un mensaje con el resultado.
 */
fun validarAcceso(
    nombre: String,
    edadTexto: String,
    contrasena: String
): String {

    // Elimina espacios al principio y al final.
    val nombreLimpio = nombre.trim()

    // Comprueba que el nombre no esté vacío.
    if (nombreLimpio.isBlank()) {
        return "Acceso denegado: introduce tu nombre"
    }

    // Convierte el texto de la edad en un número.
    // Si no es posible, devuelve null.
    val edad = edadTexto.toIntOrNull()
```

Figura 11. Primera parte de validarAcceso.

```
    // Convierte el texto de la edad en un número.
    // Si no es posible, devuelve null.
    val edad = edadTexto.toIntOrNull()

    // Comprueba que se haya introducido una edad válida.
    if (edad == null || edad < 0) {
        return "Acceso denegado: edad no válida"
    }

    // Comprueba si es mayor de edad.
    if (edad < 18) {
        return "Acceso denegado: eres menor de edad"
    }

    // Comprueba la longitud de la contraseña.
    if (contrasena.length < 8) {
        return "Acceso denegado: contraseña demasiado corta"
    }

    // Si se superan todas las comprobaciones,
    // permite el acceso.
    return "Acceso permitido. Bienvenido/a, $nombreLimpio"
}
```

Figura 12. Segunda parte de validarAcceso.

8. Conectar el botón con la funció

El botón ejecuta validarAcceso cuando el usuario pulsa COMPROBAR ACCESO. El resultado devuelto se guarda en la variable resultado, definida con mutableStateOf.

```
Button(
    onClick = {
        resultado = validarAcceso(nombre, edad, contrasena)
    },
    modifier = Modifier.fillMaxWidth()
) {
    Text("COMPROBAR ACCESO")
}
```

9. Mostrar el mensaje de respuesta

```
if (resultado.isNotEmpty()) {
    val correcto = resultado.contains(
        "Acceso permitido", ignoreCase = true
    )
    Text(
        text = if (correcto) "✅" else "❌",
        fontSize = 38.sp
    )
    Spacer(modifier = Modifier.height(8.dp))
    Text(
        text = resultado,
        color = if (correcto) Color(0xFF167A36)
                else Color(0xFFC62828),
        fontWeight = FontWeight.Bold,
        fontSize = 18.sp
    )
}
```

El mensaje se muestra solo si resultado no está vacío. Si contiene “Acceso permitido”, aparece el símbolo positivo y el texto verde. En otro caso, aparece el símbolo negativo y el texto rojo.

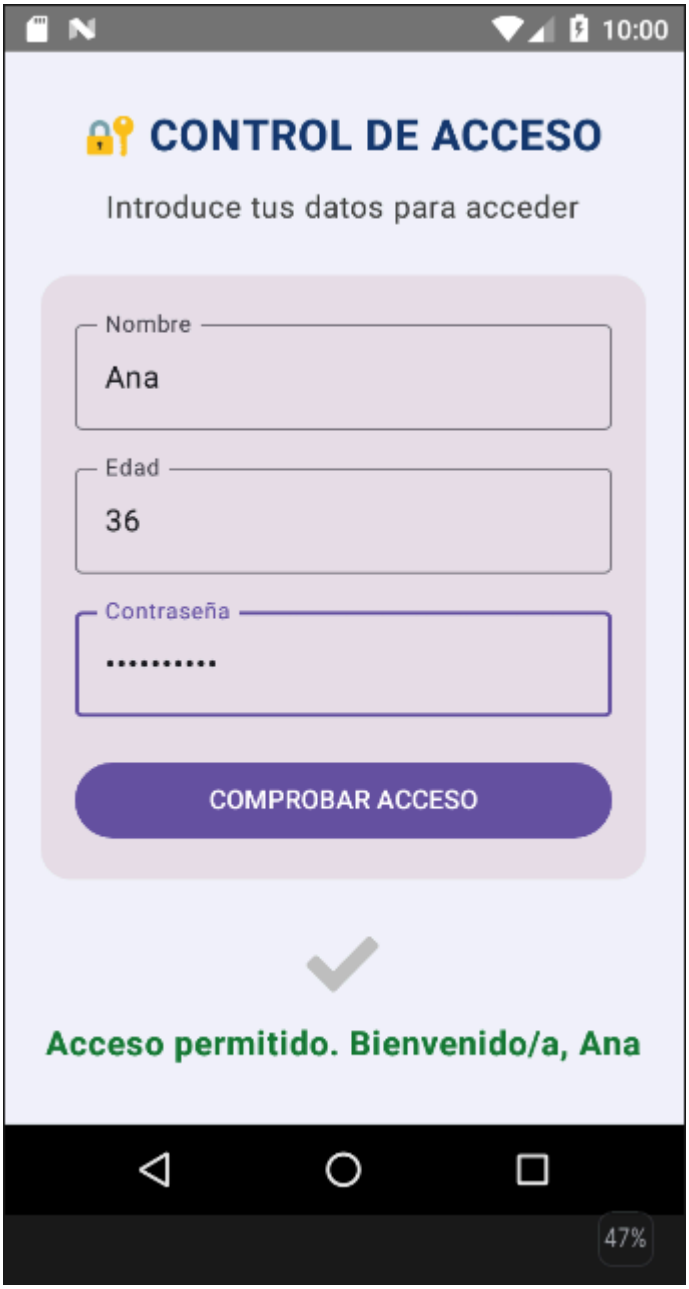


Figura 14. Condición correcto y mensaje de resultado.

10. Depuración: un error que resolvimos en clase

Durante las pruebas, el programa devolvía “Acceso permitido. Bienvenido/a, Ana”, pero la interfaz mostraba una cruz roja. El problema estaba en comparar el texto devuelto con “Acceso correcto”, una expresión distinta de la que realmente producía validarAcceso().

```
// Comparación inicial incorrecta:
resultado.contains("Acceso correcto", ignoreCase = true)

// Comparación corregida:
resultado.contains("Acceso permitido", ignoreCase = true)
```

La lección es que los mensajes utilizados para decidir el estado visual deben coincidir con los mensajes de la lógica. En aplicaciones mayores es preferible devolver un resultado estructurado, por ejemplo un Boolean o una clase de resultado, en lugar de buscar palabras dentro del mensaje.

11. Pruebas en el emulador

Nombre	Edad / contraseña	Resultado esperado
vacío	26 / 12345678	Introduce tu nombre
Ana	texto / 12345678	Edad no válida
Ana	15 / 12345678	Eres menor de edad
Ana	26 / 123	Contraseña demasiado corta
Ana	26 / 12345678	Acceso permitido

Para cada prueba, escribe los valores en el emulador, pulsa el botón y comprueba el mensaje. Anota cualquier diferencia entre el resultado observado y el esperado.

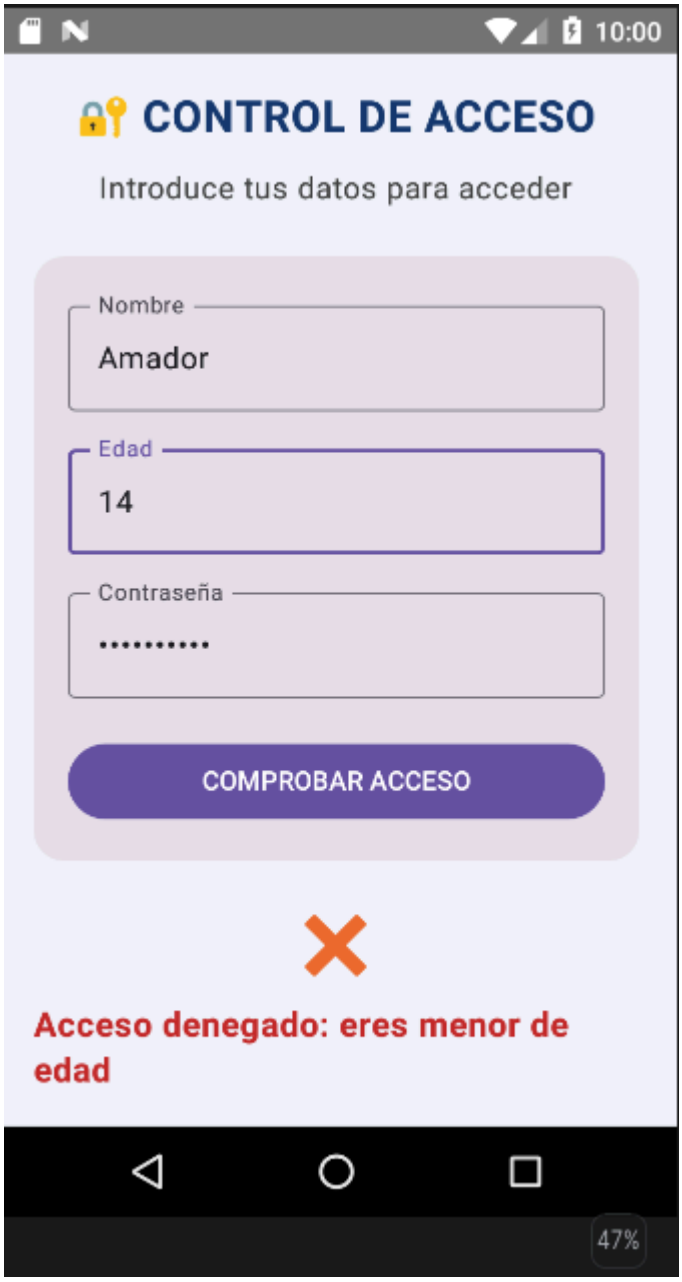


Figura 15. Prueba con acceso denegado.

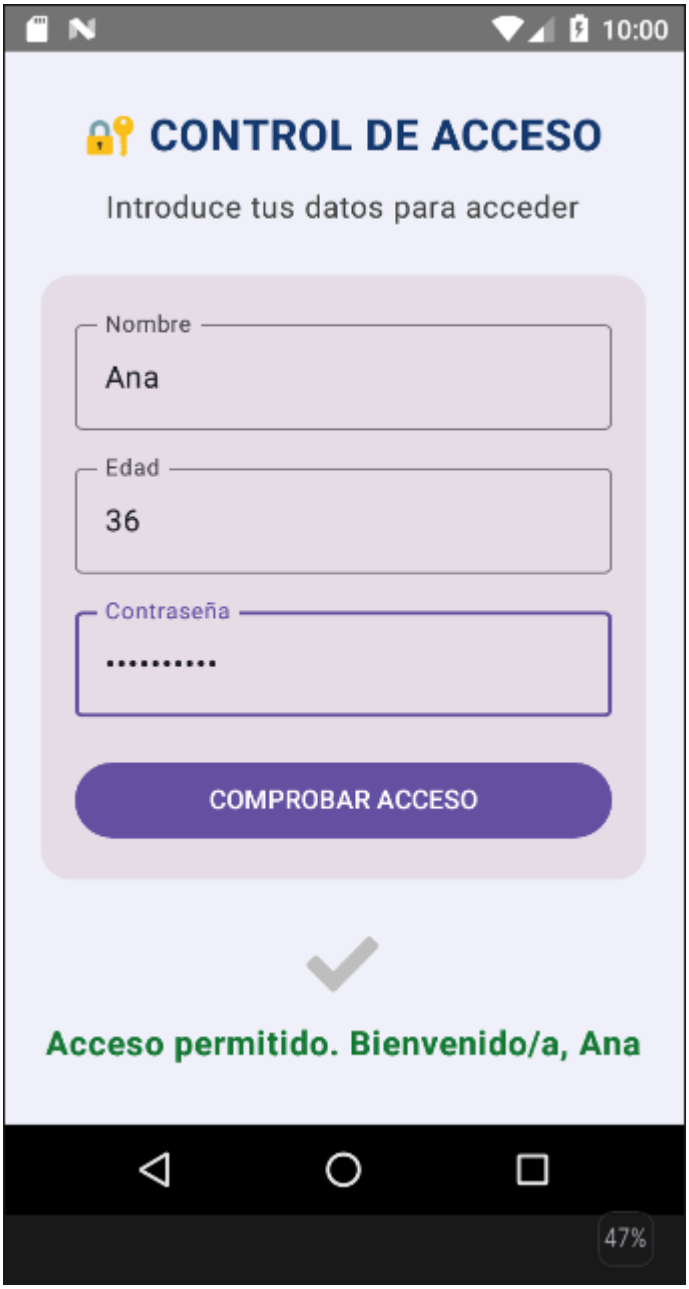


Figura 16. Prueba con acceso permitido.

12. Vista previa y herramientas de Android Studio

```
@Preview(showBackground = true)
@Composable
fun PantallaAccesoPreview() {
    Tema03composeTheme {
        PantallaAcceso()
    }
}
```

Preview muestra una vista previa de un componente sin tener que abrir el emulador. Run compila y ejecuta la aplicación real en un dispositivo. Build muestra errores de compilación, Logcat muestra registros y Debug permite detener el programa en puntos de interrupción.

PEGAR AQUÍ LA FOTOGRAFÍA 17

Función @Preview

Captura real del final de MainActivity.kt con PantallaAccesoPreview.

(Sustituye este recuadro por tu captura. No hace falta enviar otra foto si ya la tienes.)

Figura 17. Función @Preview.

13. Actividades prácticas

11. Cambia el título de la pantalla y el texto del botón.
12. Modifica validarAcceso para rechazar edades superiores a 120 años.
13. Prueba qué ocurre si el nombre contiene solo espacios.
14. Añade una comprobación para impedir contraseñas compuestas solo por espacios.
15. Explica qué archivo cambiarías para modificar la apariencia y cuál para cambiar las reglas.
16. Entrega dos capturas de funcionamiento y una breve tabla de pruebas.

14. Preguntas de repaso

17. ¿Qué diferencia hay entre Empty Activity y Empty Views Activity?
18. ¿Qué hace setContent?
19. ¿Qué significa @Composable?
20. ¿Para qué sirven remember y mutableStateOf?
21. ¿Por qué toIntOrNull es útil con un campo de texto?
22. ¿Qué diferencia existe entre ocultar visualmente una contraseña y cifrarla?
23. ¿Por qué apareció una cruz roja en una validación correcta?
24. ¿En qué se diferencian Preview, Run, Build y Logcat?

15. Entrega de la práctica

Entrega el proyecto Tema03compose con MainActivity.kt y ValidarAcceso.kt, una captura del emulador con acceso permitido, otra con acceso denegado y una tabla de al menos cinco pruebas. Verifica que el proyecto compile y que los mensajes sean coherentes con los símbolos y colores.

Guía para insertar las fotografías en este Word

Cada recuadro numerado marca el lugar exacto de una imagen. Selecciona el recuadro, bórralo y usa Insertar → Imágenes → Este dispositivo. Ajusta la imagen a un ancho aproximado de 15 cm, conserva la proporción y mantén el pie de figura. No es necesario insertar capturas en todos los recuadros si alguna pantalla no aporta información; puedes eliminarla junto con su pie.